

Project #1

Illya Starikov

Due Date: March 25, 2016

0 Introduction

- Install 2 editor plugins
- Document 5 editor features (with vim, this'll be a cinch)
- Do the job control section of project B (I use this all the time)
- What does the -exec option on find do?
- What does xargs do?
- Look up and use tmux.

1 Two Editor Plugins

For two text editors plugins, I chose (facetiously) pathogen and powerline.

1.1 pathogen.vim

Naturally when I knew I was going to install packages, my first thought was to use a package manager. Easily manipulating and deleting new packages was going to be critical, so Pathogen looked to be the proper choice.

1.1.1 What It Does

In its simplest terms, pathogen makes package management as easy as `rm -r`. My main use has been mostly to easily locate and delete new plugins that I do not use in my `bundle` directory.

1.1.2 How To Use It

After **initial setup**, pathogen actually does the rest.

1.2 Powerline

While on my quest to find text editor plugins, I stumbled upon and installed powerline. While trying to think of my second plugin to write in this report, I couldn't help but notice how much I was using Powerline.

1.2.1 What It Does

Powerline is an alternative to the standard Vim statusline.

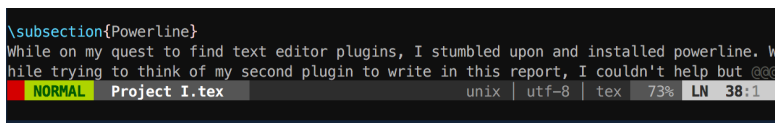


Figure 1: The Powerline In Action

It changes the configuration based on the mode that you are.

1.2.2 How To Use It

Use of powerline is automatic.

2 Five Editor Features

For my five editor features, I decided to do:

External commands (`(r ! Command)`) To run an external command outside of vim, and pipe the input back into the file. Makes inserting the date into $\text{\LaTeX}$ documents much, much faster.

Searching (`(/ ?)`) `Command F` for Vim.

Go to line (`:number`) Jumping to a specified line number.

Autocomplete (`(control+n)`) Attempts to finish the work being typed (whether it be a command or variable)

Text folding (`(z+f)a`)

3 Job Control

1. It terminates.
2. It terminates the command from the background.
3. Waits until all previous processes have finished until continueing.

4 Find `exec` ution

According to `Man`,

```
-exec utility [argument ...] ;
    True if the program named utility returns a zero value as its exit
    status. Optional arguments may be passed to the utility. The
    expression must be terminated by a semicolon (';'). If you
    invoke find from a shell you may need to quote the semicolon if the
    shell would otherwise treat it as a control operator. If the
    string '{}' appears anywhere in the utility name or the arguments
    it is replaced by the pathname of the current file. Utility will
    be executed from the directory from which find was executed.
    Utility and arguments are not subject to the further expansion of
    shell patterns and constructs.
```

Essentially `exec` allows the `exec` ution of certain programs during a command. For example, to make the use relevant to `find`, searching for an instance of a keyword in open files would `find . -exec grep KEYWORD +.`

5 *Terminal Multiplexor*

Finished, used for the job controls section (SSH to tmux to logout).

6 *Xargs*

Back to `Man`,

The `xargs` utility reads space, tab, newline and end-of-file delimited strings from the standard input and executes utility with the strings as arguments.

Any arguments specified on the command line are given to utility upon each invocation, followed by some number of the arguments read from the standard input of `xargs`. The utility is repeatedly executed until standard input is exhausted.

Spaces, tabs and newlines may be embedded in arguments using single (' ' ') or double (" ") quotes or backslashes ('\'). Single quotes escape all non-single quote characters, excluding newlines, up to the matching single quote. Double quotes escape all non-double quote characters, excluding newlines, up to the matching double quote. Any single character, including newlines, may be escaped by a backslash.

`xargs` allow for the manipulation of standard input, becoming quite useful with commands such as `find` or `ls`.

7 Conclusion

In conclusion, I have gained more familiarity with the command line. To take the assignment another step up, the entirety of this assignment has been created using the command line — Vim, `pdflatex`, `rm` for removing of $\text{\LaTeX}$ files, etc.